

Laboratorio 2 Parte 1

Descripción de la práctica:

La práctica consiste en realizar dos códigos, uno de corrección de errores y uno de detección de errores. Ambos códigos consisten en una parte de emisor y una parte de receptor. También se utilizaron dos lenguajes de programación para realizar dichos códigos, Python y en Julia. En cuanto a los códigos, el de corrección de errores era el código de Hamming y el código de detección de errores fue el de Fletcher checksum. Se realizaron pruebas para cada implementación, primero sin errores, luego con un error y finalmente con dos errores. Se espera que el código de Hamming corrija los errores cuando hay a lo más un error y se espera que el método de Fletcher detecte errores para todas las pruebas.

Pruebas realizadas:

La emisión se realizó desde Python y la recepción se realizó en Julia para ambos algoritmos.

Corrección de Hamming:

111101: mensaje enviado 0110111001

```
# Mensaje 1:
mensaje = "111101"
codificado = hamming_codificar(mensaje)
print("Mensaje original:", mensaje)
print("Mensaje codificado con Hamming:", codificado)
```

Mensaje original: 111101
Mensaje codificado con Hamming: 0110111001

```
hamming_correct("0110111001", 6, 10) | (corrected = "111101", error_pos = nothing)
hamming_correct("0110110001", 6, 10) | (corrected = "111101", error_pos = 4)
hamming_correct("0111111000", 6, 10) | (corrected = "101100", error_pos = 2)
```

1001011: mensaje enviado 00110010011

```
# Mensaje 2:
mensaje = "1001011"
codificado = hamming_codificar(mensaje)
print("Mensaje original:", mensaje)
print("Mensaje codificado con Hamming:", codificado)
```

Mensaje original: 1001011
Mensaje codificado con Hamming: 00110010011

```

hamming_correct("00110010011", 7, 11) | (corrected = "1001011", error_pos = nothing)
hamming_correct("00110000011", 7, 11) | (corrected = "1001011", error_pos = 4)
hamming_correct("00010010010", 7, 11) | (corrected = "0001000", error_pos = 6)

```

1000111: mensaje enviado 10110000111

```

# Mensaje 3:
mensaje = "1000111"
codificado = hamming_codificar(mensaje)
print("Mensaje original:", mensaje)
print("Mensaje codificado con Hamming:", codificado)

```

Mensaje original: 1000111
Mensaje codificado con Hamming: 10110000111

```

hamming_correct("10110000111", 7, 11) | (corrected = "1000111", error_pos = nothing)
hamming_correct("10110010111", 7, 11) | (corrected = "1000111", error_pos = 4)
hamming_correct("10110000001", 7, 11) | (corrected = "1010001", error_pos = 3)

```

Detección Fletcher Checksum:

111101: checksum (8 bits) 1111010011110100

```

# Mensaje 1:
mensaje = "111101"
bloque_bits = 8
resultado = fletcher_checksum_bloques(mensaje, bloque_bits)

print("Mensaje original:", resultado["mensaje_original"])
print("Checksum (Fletcher):", resultado["checksum"])
print("Mensaje codificado con Checksum (Fletcher):", resultado["mensaje_codificado"])

```

Mensaje original: 111101
Checksum (Fletcher): 1111010011110100
Mensaje codificado con Checksum (Fletcher): 1111011111010011110100

```

check_fletcher8("111101", parse(Int, "1111010011110100", base=2)) | "11110100"
check_fletcher8("111111", parse(Int, "1111010011110100", base=2)) | "Error"
check_fletcher8("110111", parse(Int, "1111010011110100", base=2)) | "Error"

```

1001011: checksum (16 bits) 10010110000000001001011000000000

```
Mensaje original: 1001011
Checksum (Fletcher): 10010110000000001001011000000000
Mensaje codificado con Checksum (Fletcher): 100101110010110000000001001011000000000
```

```
check_fletcher16("1001011", parse(Int, "10010110000000001001011000000000", base=2)) "1001011000000000"
check_fletcher16("1001111", parse(Int, "10010110000000001001011000000000", base=2)) "Error"
check_fletcher16("1001111", parse(Int, "10010110000000001011011000000000", base=2)) "Error"
```

[illegible][illegible]

- Si es posible manipular los bits para provocar que el algoritmo no detecte un error. El ejemplo más sencillo es el trivial de una sucesión de ceros:

- El algoritmo de corrección de Hamming presenta la ventaja de que puede corregir si existe un error en el mensaje. Sin embargo, es bueno corrigiendo cuando solo hay un error. Para dos o más errores realiza correcciones que no coinciden con el mensaje original a pesar de ser capaz de identificar que hubo algún error. Esto se debe a que la corrección es bastante compleja y puede que no sea muy fiable en el caso de considerable ruido. El algoritmo de detección de Fletcher checksum tiene la ventaja de que es bastante fácil de implementar y es bastante rápido, pues solo consiste en operaciones aritméticas. Resulta bastante bueno detectando errores en los mensajes, incluso cuando es más de uno. Sin embargo, es cierto que es imposible identificar cual o dónde es el error.

Repositorio: https://github.com/ManuelAlejandroMartinezFlores/Redes_Lab2